

Lecture 14: Data Cleaning

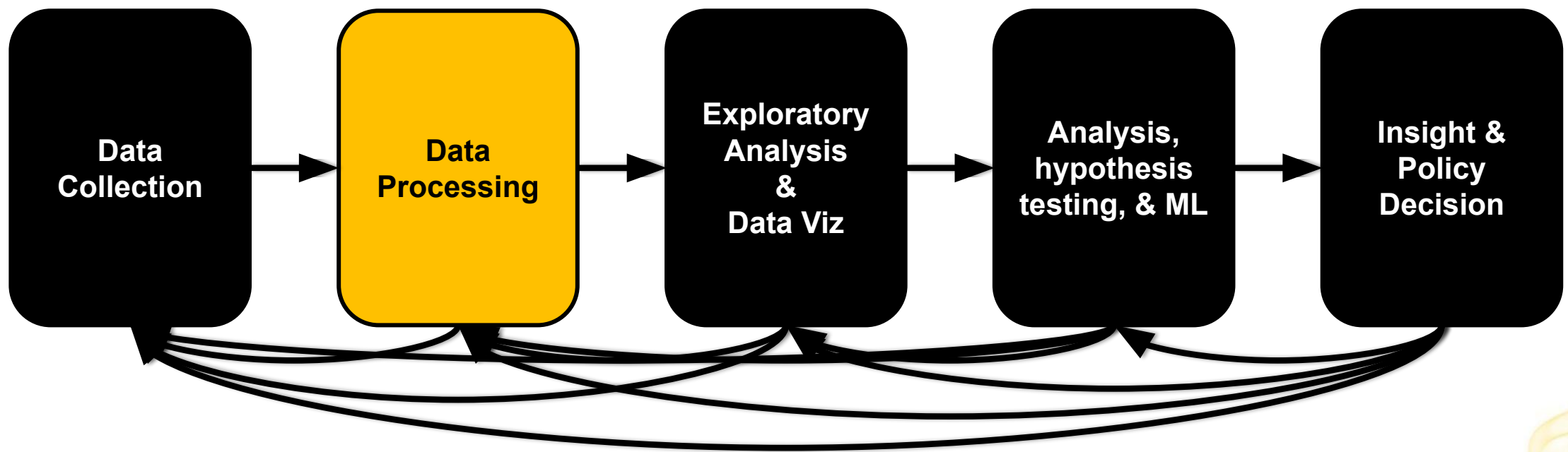
CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA4, WA4 out
 - Mid-semester check-in assignment available this week
 - Also asks about office hour preferences
 - Spring break next week!
- 

Data Pre-Processing



Data Pre-Processing

a.k.a. Data Processing, or the step where you:

- Clean, scrub, and sanity check data to ensure **data quality**

Recall that poor data quality may lead to **unexpected results** or **misleading information**

- Duplicates
 - Formatting
 - Weird outliers
 - Mistakes
 - Transformations needed
 - Sampling
- 

Data Cleaning

Data Cleaning

Data cleaning is the process of fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset



"This is not what I meant when I said 'we need better data cleansing!'"

Why do we need Data Cleaning?

We often get data that is “**messy**”

- Missing values
- Weird outliers
- Columns that need to be combined
- Multiple tables need to be joined
- Inconsistency:
 - Ex. Mixed date format usage
 - "MM/DD/YYYY," "YYYY-MM-DD," and "DD-Mon-YY"
 - 01/15/2023, 2023-01-15, Jan-15-23

These can impede and affect our analysis + results!

Looking for issues

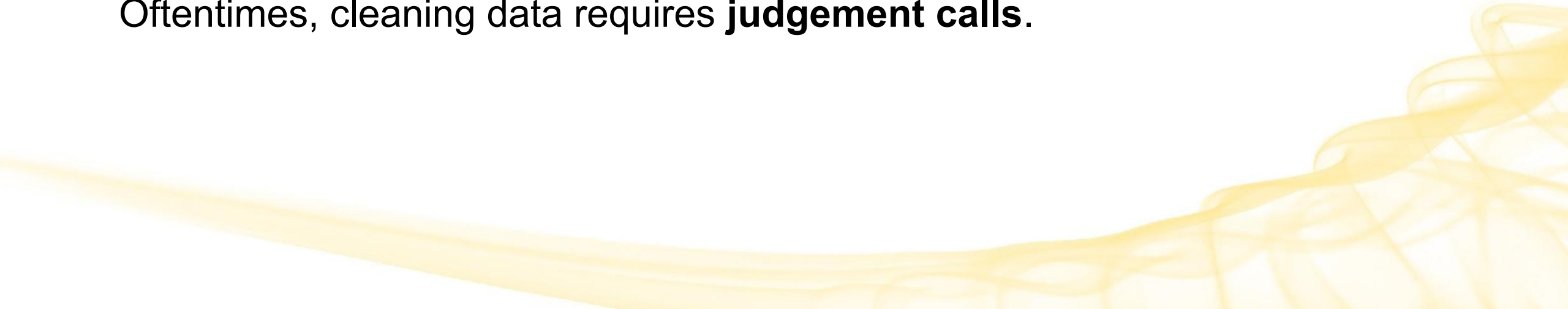
Sometimes, it's **simple** and **obvious**

- Maybe a word is misspelled in a categorical variable
- You can fix this using `apply()` in 🐼 Pandas

Sometimes it's **non-obvious**!

- Not all data cleaning issues are immediately apparent or easy to solve.

Oftentimes, cleaning data requires **judgement calls**.

A decorative yellow swirl graphic is located in the bottom right corner of the slide, extending from the bottom edge and curving upwards and to the right.

Where to start

Look to see what you have.



Pandas has some nice features to help us do that!

```
print('General info:')
df.info()
```

General info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 150 entries, 0 to 149

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Id	150 non-null	int64
1	SepalLengthCm	150 non-null	float64
2	SepalWidthCm	150 non-null	float64
3	PetalLengthCm	150 non-null	float64
4	PetalWidthCm	150 non-null	float64
5	Species	150 non-null	object

dtypes: float64(4), int64(1), object(1)

memory usage: 7.2+ KB

```
print("Shape of dataset")
print(df.shape)
```

```
print('-> This dataset has', df.shape[0], 'rows and', df.shape[1], 'columns')
```

Shape of dataset
(150, 6)

-> This dataset has 150 rows and 6 columns

```
print('Data Types:')
print(df.dtypes)
```

Data Types:

Id	int64
SepalLengthCm	float64
SepalWidthCm	float64
PetalLengthCm	float64
PetalWidthCm	float64
Species	object
dtype:	object

```
print('Descriptives stats:')
print(df.describe())
```

Descriptives stats:

	Id	SepalLengthCm	SepalWidthCm
count	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000
std	43.445368	0.828066	0.433594
min	1.000000	4.300000	2.000000
25%	38.250000	5.100000	2.800000
50%	75.500000	5.800000	3.000000
75%	112.750000	6.400000	3.300000
max	150.000000	7.900000	4.400000

Where to start

We need to know if there are any potential issues in the data.



Pandas has some nice features to help us do that!

```
print("Null values:")  
df.isnull().sum()
```

Null values:

0

Id	0
SepalLengthCm	0
SepalWidthCm	0
PetalLengthCm	0
PetalWidthCm	0
Species	0

```
print('Duplicates:')  
print(df[df.duplicated()])
```

Duplicates:

Empty DataFrame

Columns: [Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species]

Index: []

The Easy Stuff

Data Typing

Sometimes the data is in the **wrong format!**

What to do:

- **Date-Time** data (very common)
 - 🐼 Pandas offers a robust [datetime library](#)
- **Other data types**
 - Use `df[COLUMN].astype(SOME_TYPE)` for straightforward conversions
- **More complex issues**
 - Utilize `df[COLUMN].apply(conversion_function)`

Combining and Merging Data Sources

Often you need to **merge various tables/datasets**.

What to do:

- Make sure the **file formats all match**.
 - Check that columns have consistent data types, matching column names
 - **This can be tricky sometimes.**
 - Various data format issues, resolving **data conflicts**, and deciding on the **appropriate type of join** (ex. inner join, outer join) to use when merging tables
- Remember to **sanity check** your work!

Evolving Labeling Schemes

Situations where the labels or categories used to classify data change over time.

This happens when how things are labeled changes halfway through.

Example: one field **gets split** into two subfields, or people decide it makes more sense to **change how something is recorded midway** through.

- **Example Scenario:** Initially, a column that typically records "**failure**" is **modified midway through the project** to include "**catastrophic failure**" and "**partial failure**".

You'll have to cope with these as best you can (**Coping strategies**)

- Divide the dataset in two (based on the time of change)?
- Infer old ratings (based on available data)?
- Count both "catastrophic failure" and "partial failure" as just "failure"?

Example: evolving labeling scheme for product quality ratings.

SCENARIO:

Suppose you are **analyzing customer reviews** for a product over a period of time, and you **notice a change** in the way product quality is rated during the analysis.

Initial Labeling Scheme (Phase 1):

Date	Product_ID	Customer_Name	Quality_Rating
2022-01-05	101	Alice	Good
2022-02-10	102	Bob	Excellent
2022-03-15	103	Carol	Fair

Example: evolving labeling scheme for product quality ratings.

Midway through the project, the company decided to **change the labeling scheme** to include more detailed quality ratings, introducing new subcategories such as "Excellent," "Very Good," "Good," "Fair," and "Poor".

Evolving Labeling Scheme (Phase 2):

Date	Product_ID	Customer_Name	Quality_Rating
2022-04-20	104	Dave	Very Good
2022-05-25	105	Eve	Good
2022-06-30	106	Frank	Excellent
2022-07-05	107	Grace	Fair
2022-08-10	108	Henry	Poor

Example: evolving labeling scheme for product quality ratings.

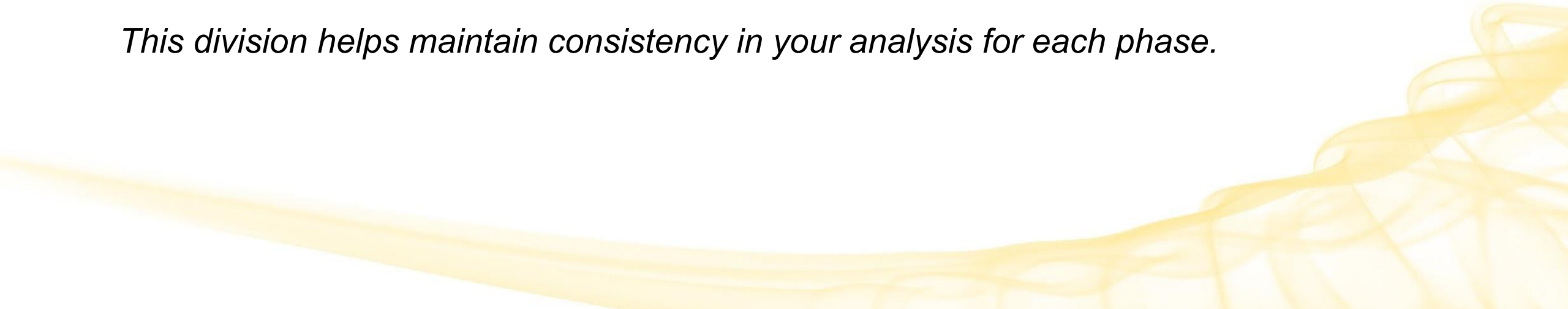
Possible Coping Strategies:

Option 1: Splitting the Dataset (by Phases)

To accommodate changes in the labeling scheme, consider **dividing the dataset** into two separate groups:

1. One containing data from the initial phase (Phase 1).
2. Another for the data collected after the labeling scheme change (Phase 2).

This division helps maintain consistency in your analysis for each phase.



Example: evolving labeling scheme for product quality ratings.

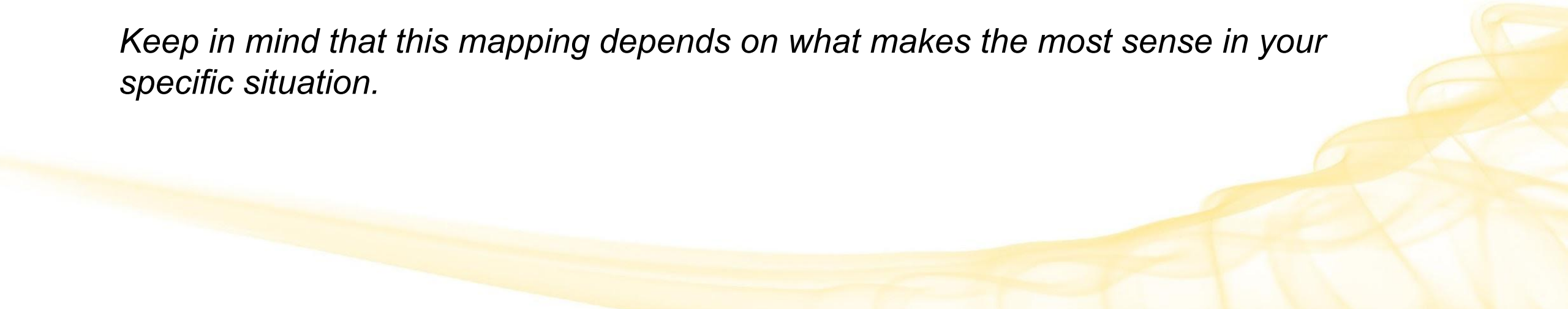
Possible Coping Strategies:

Option 2: Infer/ Interpret Old Ratings

For records in Phase 1, where you only have "Good," "Excellent," and "Fair" ratings, you can infer how they might map to the new labeling scheme.

- Ex. you might consider mapping "Good" to "Very Good" and "Fair" to "Fair" in the updated scheme.

Keep in mind that this mapping depends on what makes the most sense in your specific situation.



Duplicate Records

Duplicated Records

Sometimes people put a bunch of **duplicate records** in your system!

- If they are **exact duplicates**: maybe just do `df.drop_duplicates()`
- If they are duplicates where **some of them are subtly different**: identify the ones that are the true values and drop the rest

ALWAYS CHECK FOR THIS



Duplicates Example

```
df = pd.read_csv('dups.csv')
df.head(5)
```

	ID	Name
0	1	John Doe
1	2	Alice Smith
2	3	Bob Johnson
3	4	joHn dOe
4	5	Alice Smith



```
# Convert names to lowercase and remove leading/trailing/extra spaces
df['Clean_Name'] = df['Name'].str.lower().str.strip().str.replace('\s{2,}', ' ', regex=True)
print(df)
```

	ID	Name	Clean_Name
0	1	John Doe	john doe
1	2	Alice Smith	alice smith
2	3	Bob Johnson	bob johnson
3	4	joHn dOe	john doe
4	5	Alice Smith	alice smith

```
# Identify and keep unique records based on the cleaned names
clean_df = df.drop_duplicates(subset='Clean_Name', keep='first').drop(columns='Clean_Name')
print(clean_df)
```

	ID	Name
0	1	John Doe
1	2	Alice Smith
2	3	Bob Johnson

Outlier Detection

What defines an outlier?

The formal definition of an outlier may vary depending on the context and statistical method employed.

One common approach is to identify outliers as **data points that deviate significantly from the mean**, often defined as being several standard deviations away.

Outliers

Outliers are data points that **significantly deviate from the majority of the data in a dataset.**

- Example: unusually high or low values that may be indicative of errors, anomalies, or rare events.

Detecting outliers is important because they can **skew statistical analyses and machine learning models.**



Outlier Detection: Common approaches

You can find outliers multiple ways. Looking for **extreme z-scores** is one way

$$Z = \frac{x - \mu}{\sigma}$$

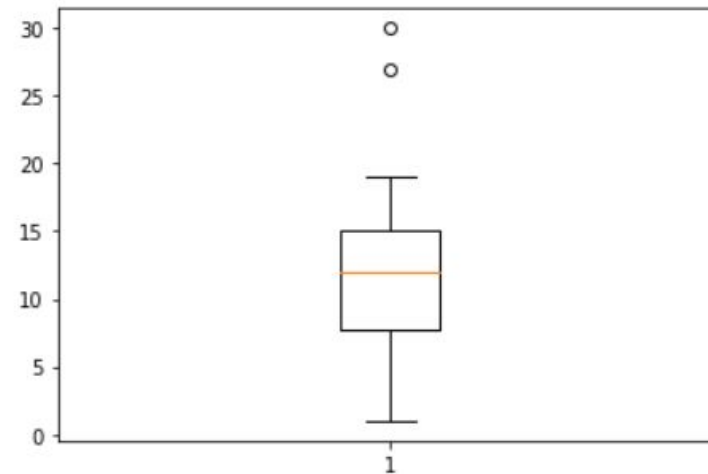
Z = standard score

x = observed value

μ = mean of the sample

σ = standard deviation of the sample

A simple box and whisker plot is another



Outlier Detection using Z-Scores

Z-scores are a statistical measure that quantifies how far a data point is from the mean (average) of a dataset in terms of standard deviations.

- A **high positive or negative z-score** suggests that a data point is far from the mean and is a potential outlier.

$$Z = \frac{x - \mu}{\sigma}$$

Z = standard score

x = observed value

μ = mean of the sample

σ = standard deviation of the sample

Outlier Detection using Z-Scores

Example:

Test Scores: [80, 85, 88, 90, 92, 95, 98, 100, 150]

Step 1: Calculate the Mean and Standard Deviation:

- Mean (μ) = $(80 + 85 + 88 + 90 + 92 + 95 + 98 + 100 + 150) / 9 = \mathbf{100.44}$
- Standard Deviation (σ) $\approx \mathbf{21.11}$ (rounded for simplicity)

$$Z = \frac{x - \mu}{\sigma}$$

Z = standard score

x = observed value

μ = mean of the sample

σ = standard deviation of the sample

Outlier Detection using Z-Scores

Example:

Test Scores: [80, 85, 88, 90, 92, 95, 98, 100, 150]

Step 2: Calculate Z-scores for each test score using the formula:

- For the test score of 150:
 - $Z = (150 - 100.44) / 21.11 \approx 2.35$
- For the other scores, calculate their respective z-scores.

$$Z = \frac{x - \mu}{\sigma}$$

Z = standard score

x = observed value

μ = mean of the sample

σ = standard deviation of the sample

Outlier Detection using Z-Scores

Example:

Test Scores: [80, 85, 88, 90, 92, 95, 98, 100, 150]

Step 3: Identify Extreme Z-scores:

Common threshold used for an extreme z-score:

“greater than 2 or less than -2” suggests that a data point is significantly far from the mean.

- Here, the z-score for the score of 150 is approximately 2.35, which is greater than 2.
 - Indicates that the score of 150 is significantly above the mean and is a potential outlier.

$$Z = \frac{x - \mu}{\sigma}$$

Z = standard score

x = observed value

μ = mean of the sample

σ = standard deviation of the sample

Should we always remove an outlier?



It depends!

Ultimately, when doing a data science project, you have some goal in mind. Remove the outlier if it hurts that goal.

Consider:

Context of the Outlier

Is the outlier an **unusual event that will likely never happen again?**

- Something like “light intensity during an eclipse”

→ If it is, removing the outlier may be appropriate.

Impact on Model Performance

Is the outlier something that **will actually hurt your model?**

- You probably don’t need a model factoring in Jeff Bezos when doing financial transactions

Relevance

Is the outlier something that is **a core element of your data?**

- Profits during a sale, wait times during traffic
- We shouldn’t remove without carefully considering.

Recap

Outlier data is a numeric value that is **much larger or smaller than other values** in the same feature. Outlier data is usually defined as two or three standard deviations from the feature mean.

Duplicate data are **two or more identical instances** in a dataset. Duplicate instances are usually erroneous and should be removed.



Missing Data

Missing Data

In data cleaning, “missing data” refers to any instances within a dataset where certain values or observations are **not recorded or are incomplete**.

- In most databases, missing data is represented as **NULL**
- In Python, missing data is represented as:
 - **NaN** (not a number)
 - **NaT** (not a time)
 - **None** (an unspecified object, Python’s version of null)
 - **a blank value** (ex. an empty string).
- 🐼 Pandas has some handy methods for detecting missing values
 - Ex. `df.isna().isany()` and `df.isna().sum()` are useful

Dirty Data

Missing, outlier, and duplicate data are collectively called dirty data.

	Country	Continent	SurfaceArea	Population	LifeExpectancy	IndependenceYear	
0	Antarctica	Antarctica	13120000.0	0	NaN	NaT	
1	China	Asia	9572900.0	1277558000	71.4	-1523	Outlier
2	Bangladesh		143998.0	129155000	60.2	1971	
3	Bolivia	South America	NaN	8329000	63.7	1825	
4	Switzerland	None	41284.0	7160400	79.6	1499	
5	United States	North America	9363520.0	278357000	77.1	1776	
6	United States	North America	9363520.0	278357000	77.1	1776	Duplicate

Diagram annotations:

- Missing - unknown**: Points to the 'Continent' column for rows 1, 2, and 3.
- Missing - inapplicable**: Points to the 'LifeExpectancy' and 'IndependenceYear' columns for row 0.
- Outlier**: Points to the 'IndependenceYear' value of -1523 for China (row 1).
- Duplicate**: Points to the entire row for the United States (row 6), which is identical to row 5.

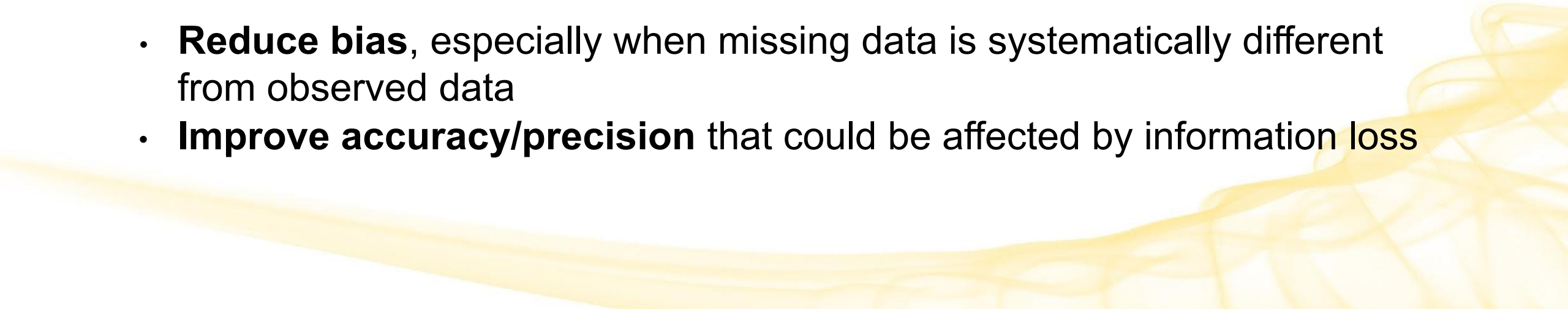
Dirty data creates bias and inefficiencies in data analysis.

Why do we care about missing data?

Understanding the representation of missing values in your dataset is critical for:

- Proper data cleaning/pre-processing
- Ensuring models (*future topic*) perform well

Taking missing data into account will help you:

- **Reduce bias**, especially when missing data is systematically different from observed data
 - **Improve accuracy/precision** that could be affected by information loss
- 

Types of Missing Data (called “Mechanisms”)

Missing completely at random (MCAR)

Missing at random (MAR)

Missing not at random (MNAR)

Types of Missing Data (called “Mechanisms”)

Missing completely at random (MCAR)

- There may not be a pattern as to why the value is missing.
- The reason is unrelated to the dataset.
 - Ex. a weighing scale runs out of batteries

Missing at random (MAR)

- The value is missing due to other data.
- The reason can be described by data in another variable in the dataset.
 - Ex. women specifically under-report their education level

Missing not at random (MNAR)

- The missing value is related to the value itself.
- The missing value is not dependent on other variables in the dataset.
 - Ex. people who have had a drug addiction are less likely to respond to a drug-related question on a general survey

More than one thing could be happening to your column!

MCAR Example

Age	Gender	Income	Favorite Color
25	Male		Blue
30	Female	50,000	Red
22	Male	45,000	Red
28	Female	60,000	Green
	Male	55,000	Yellow

The missing data (Income and Age) happened randomly due to technical issues (some survey forms didn't save properly) and has no connection to the observed data (Age, Gender, Favorite Color) or the actual missing values.

MAR Example

In a medical study, older patients are more likely to skip questions about their mental health status, but their actual mental health condition does not affect whether they skip the question.

Age	Gender	Physical Health Score	Mental Health Status
65	Female	80	
70	Male	75	Mild Depression
30	Female	90	No Issues
50	Male	85	
45	Female	88	Moderate Anxiety

Reason for Missingness: Older age is associated with the likelihood of missing data on Mental Health Status, but the actual Mental Health Status is not directly causing the missingness.

Implication: The missing data is related to observed data (Age), but not to the missing data itself (Mental Health Status).

MNAR Example

In a financial survey, people with very high incomes are less likely to report their income because they prefer to keep it confidential.

Age	Gender	Occupation	Income
25	Male	Engineer	70,000
30	Female	Doctor	
22	Male	Student	20,000
28	Female	Lawyer	
45	Male	Executive	

***Reason for Missingness:** The actual value of the income **itself influences** the likelihood of the data being missing.

*** Implication:** The missing data is directly related to the value of the missing data itself.



How do we handle missing data?

Handling Data Missing at Random (MAR)

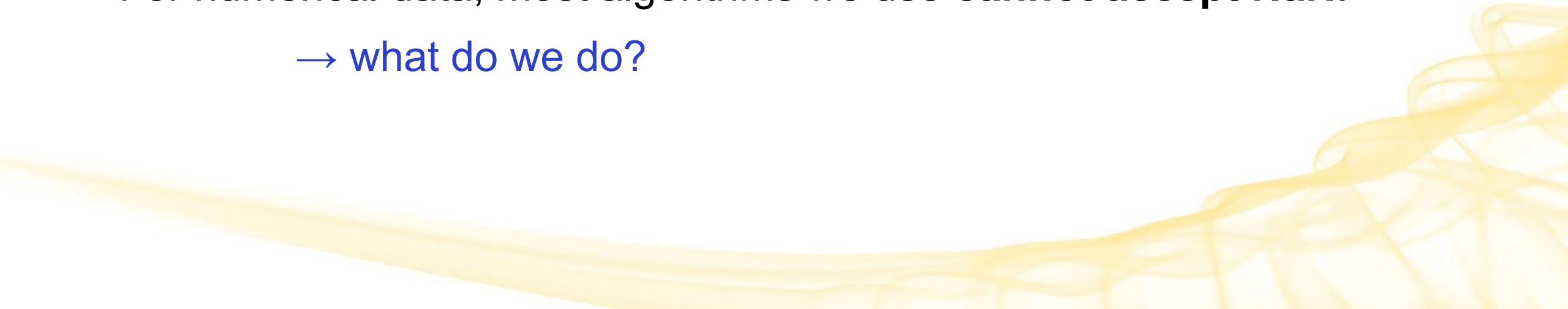
For data that is **categorical**, it's fine! "Missing" can just become a new category.

- Ex: If our category is college major, we could have:

`{ "Computer Science", "Philosophy", "Did not answer" }`

For numerical data, most algorithms we use **cannot accept NaN**.

→ what do we do?



Solution 1: Drop it

You can delete rows or columns with missing values.

- Example: 1% of your data has rows with missing values and you have a lot of data (a terabyte), so you drop all rows that have missing values.
- 🐼 Pandas
 - `df.dropna()`
 - ex. drop all rows (observations) that are missing values in the “Name” column
 - `df.drop()`
 - ex. delete the entire “Name” column

Listwise Deletion: Remove any rows with missing data. This works well when the proportion of missing data is *small*.

Pairwise Deletion: Use only the data points where both variables being analyzed (ex. income and education) have non-missing values, excluding cases with missing values for either variable from that specific analysis.

```
>>> df
   name      toy      born
0  Alfred    NaN      NaT
1  Batman  Batmobile  1940-04-25
2 Catwoman  Bullwhip    NaT
```

Drop the rows where at least one element is missing.

```
>>> df.dropna()
   name      toy      born
1  Batman  Batmobile  1940-04-25
```

Drop the columns where at least one element is missing.

```
>>> df.dropna(axis='columns')
   name
0  Alfred
1  Batman
2 Catwoman
```

Drop the rows where all elements are missing.

```
>>> df.dropna(how='all')
   name      toy      born
0  Alfred    NaN      NaT
1  Batman  Batmobile  1940-04-25
2 Catwoman  Bullwhip    NaT
```

Keep only the rows with at least 2 non-NA values.

```
>>> df.dropna(thresh=2)
   name      toy      born
1  Batman  Batmobile  1940-04-25
2 Catwoman  Bullwhip    NaT
```

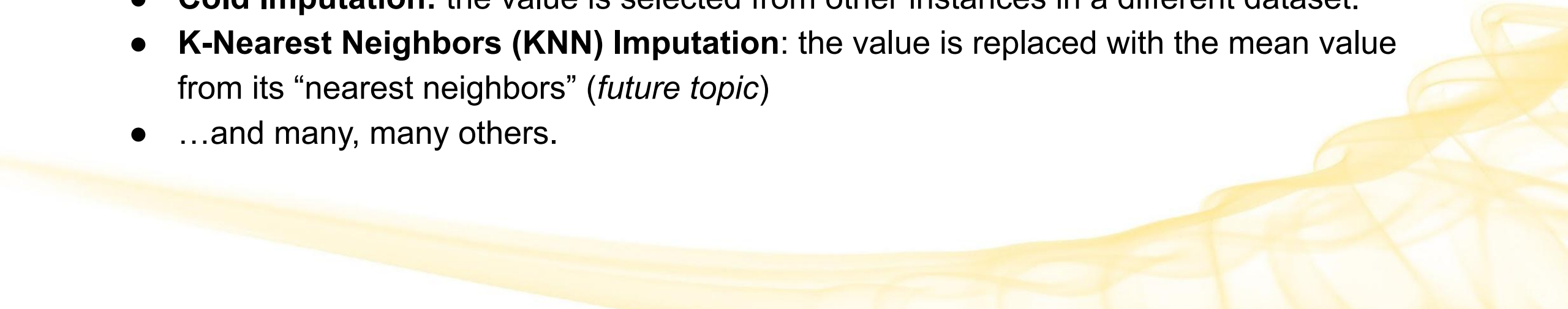
Define in which columns to look for missing values.

```
>>> df.dropna(subset=['name', 'toy'])
   name      toy      born
1  Batman  Batmobile  1940-04-25
2 Catwoman  Bullwhip    NaT
```

Solution 2: Apply Imputation!

Imputation is the process of **replacing missing values with estimates** (new values).

Data may be imputed in several ways:

- **Mean/Median/Mode Imputation:** Imputation using descriptive statistics-based values for that particular variable
 - **Hot Imputation:** the value is selected from other instances in the same dataset.
 - **Cold Imputation:** the value is selected from other instances in a different dataset.
 - **K-Nearest Neighbors (KNN) Imputation:** the value is replaced with the mean value from its “nearest neighbors” (*future topic*)
 - ...and many, many others.
- 

Imputation Using Mean/Median/Mode

Quick and easy, BUT can introduce bias if the missing data is not randomly distributed. Illustrative, but can be a naïve method.

- **Technique:**
 - **Calculate the mean/median (sometimes mode) of the non-missing values** in a column, then replace the missing values within each column separately and independently from the others with the calculated value.
- **Remember:**
 - **This can only be used with numeric data.**
 - Only suitable for continuous and discrete numerical variables.

Example: Mean Imputation

Mean Imputation: Set the value to the mean of that column

Example:

	col1	col2	col3	col4	col5			col1	col2	col3	col4	col5	
0	2	5.0	3.0	6	NaN	mean() →		0	2.0	5.0	3.0	6.0	7.0
1	9	NaN	9.0	0	7.0			1	9.0	11.0	9.0	0.0	7.0
2	19	17.0	NaN	9	NaN			2	19.0	17.0	6.0	9.0	7.0

Example: Mode Imputation

Mode Imputation: Fill in with the most common value

Example:

Make
Ford
Ford
Fiat
BMW
Ford
Kia
Fiat
Ford
Kia

Mode = Ford



Price
Ford
Ford
Fiat
BMW
Ford
Kia
Ford
Fiat
Ford
Ford
Kia

When to use Mean vs. Median vs. Mode

- **Mean:** Prefer if data is **numeric** and *not* skewed.
- **Median:** Prefer if data is **numeric** and **skewed**.
- **Mode:** Prefer if the data is a **string**(object).
 - Suitable for both string and numeric, but less commonly used for numeric data.
 - Typically more appropriate for categorical data or situations where there's a clear mode in the distribution.
 - Can also use a new category (“unknown”) for categorical data

Use Mean/Median/Mode Imputation when **no more than 5%** of the variable contains missing data.

Hot-Deck Imputation

A method used to fill in missing values in a dataset by replacing them with **values from similar records/rows**.

How does it work? Find **a row that is most similar** to the one with the missing value, and copy that value.

- Ex. If someone is missing a grade from 320, find a student who has taken all the same classes as them and copy over their 320 grades.

→ Any potential pitfalls?

Hot-Deck Imputation Example

Suppose you have a dataset containing information about the ages of individuals, but some ages are missing

ID	Gender	Age
101	Male	32
102	Female	NaN
103	Male	NaN
104	Female	45
105	Male	NaN
106	Female	28

Hot-Deck Imputation Example

Identify Similar Cases

Determine which cases (rows) are similar (such as gender or other relevant characteristics) or close to the one with missing data.

Impute with Similar Values

For each missing age value, find a similar individual (a "donor") from the dataset. In this example, let's use the same gender as a criterion for similarity.

ID	Gender	Age
101	Male	32
102	Female	NaN
103	Male	NaN
104	Female	45
105	Male	NaN
106	Female	28

Hot-Deck Imputation Example

After applying hot-deck imputation, your table might look like this

		Before	After
ID	Gender	Age	Age
101	Male	32	32
102	Female	NaN	28
103	Male	NaN	32
104	Female	45	45
105	Male	NaN	32
106	Female	28	28

Incorrect Data

What does it mean for data to be incorrect?



Types of Incorrect Data

- People lied to you
 - Your instrument broke
 - You've been recording the wrong metrics
 - You have two identical entries with different values for one column
 - Illegal values
 - Unclear default values
- 

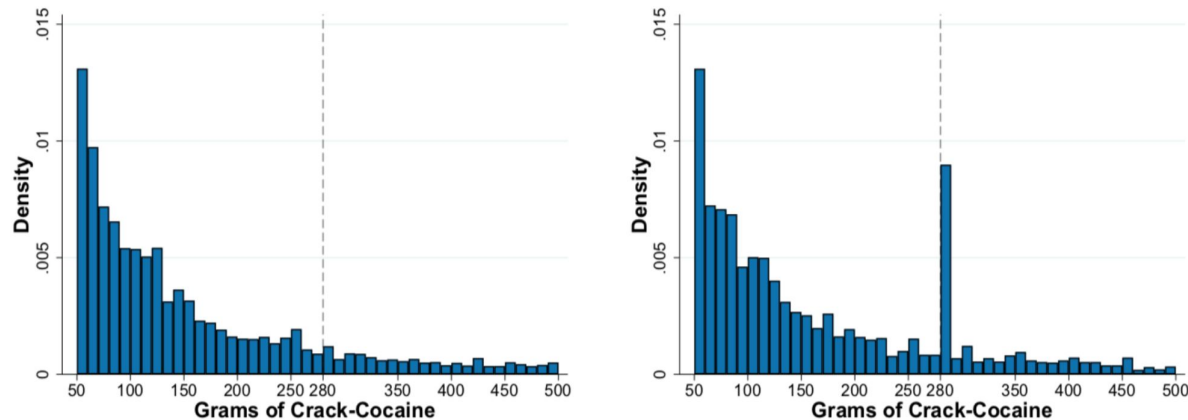
Detecting Incorrect Data

Detecting incorrect data involves **looking for anomalies** or patterns that deviate from what's expected or reasonable in your dataset.



Detecting Incorrect Data

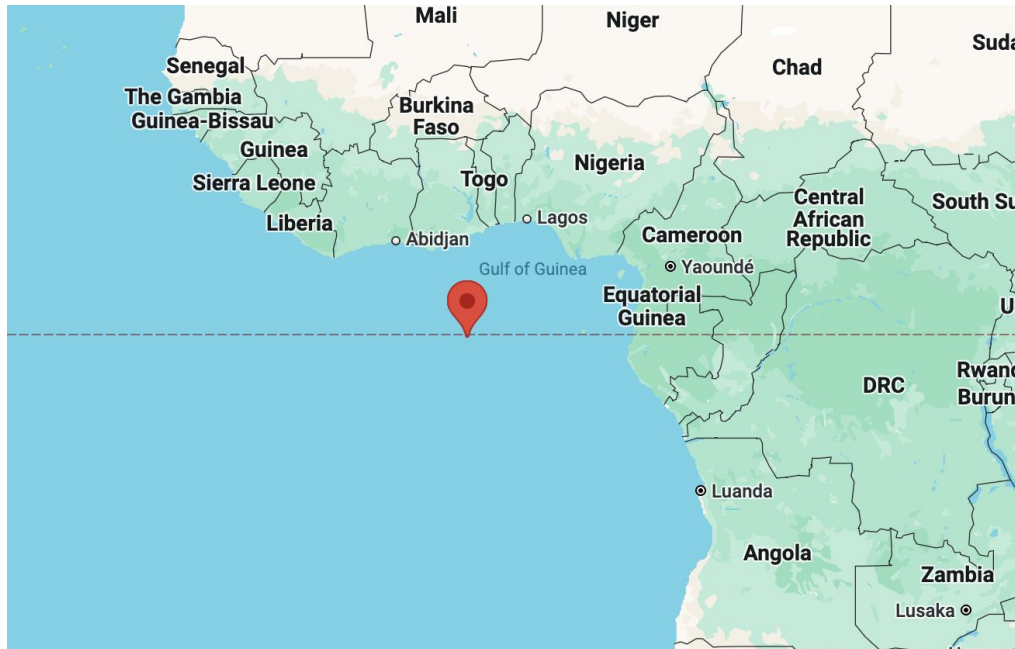
Figure 1. Changing Distribution of Drug Amounts Around 280g Pre- and Post-2010.
(a) 1999-2010 (b) 2011-2015



- **Attractors:** Check for unusual spikes or concentrations of data points in specific categories or values.
- **Discontinuities:** Look for abrupt changes/shifts or discontinuities in your data.
 - Ex. Once the limit for how much marijuana constitutes a felony changed, there was a significant discontinuity centered around that threshold (people might adjust their behavior based on the new legal limit).

https://econweb.umd.edu/~tuttle/files/tuttle_mandatory_minimums.pdf

Detecting Incorrect Data



- **Modes that don't make sense:** Identify modes (frequent values) in your data that don't align with the expected patterns.
 - Examples:
 - While analyzing geographical data, finding many people living at latitude/longitude 0,0
 - Large amount of default timestamps
 - 2000-01-01 00:00:00 - start of the millenium
 - January 1st 1970 00:00:00 UTC - Unix time origin
 - January 1st 1900 - default date used by Microsoft Excel
 - Huge number of *unexpected* zeros

Detecting Incorrect Data

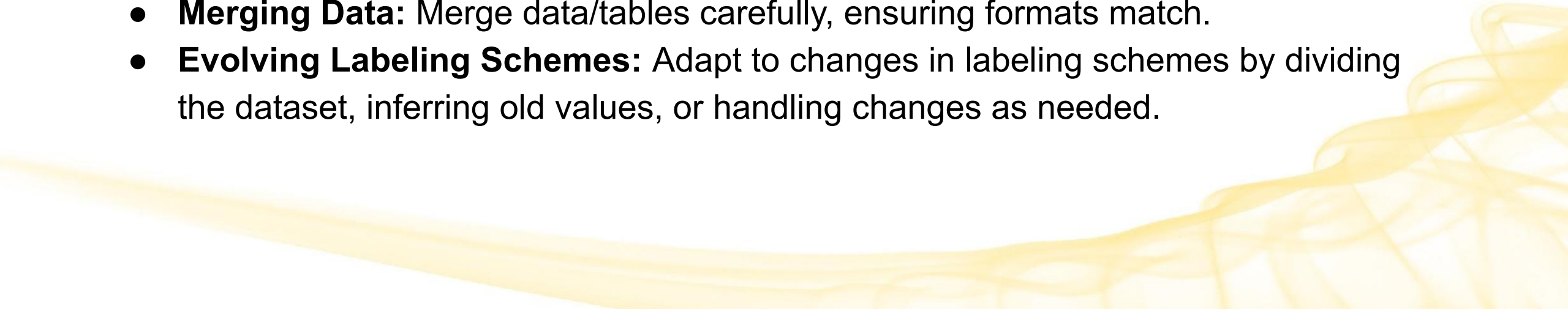


Boundary Conditions

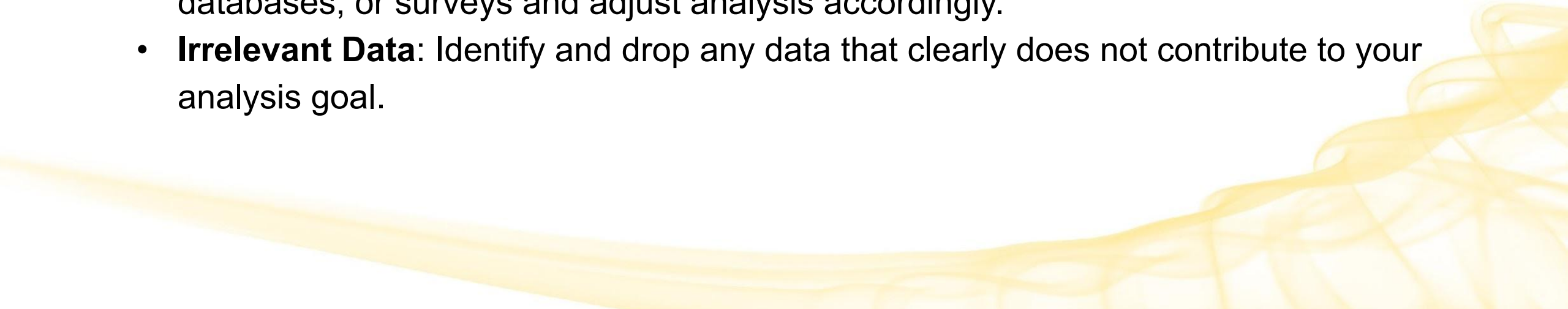
- **Data outside valid bounds:** Check for data points that fall well outside the valid range or bounds of a variable.
 - Ex. People who have been playing video games for one million hours

Summing up Data Pre-Processing

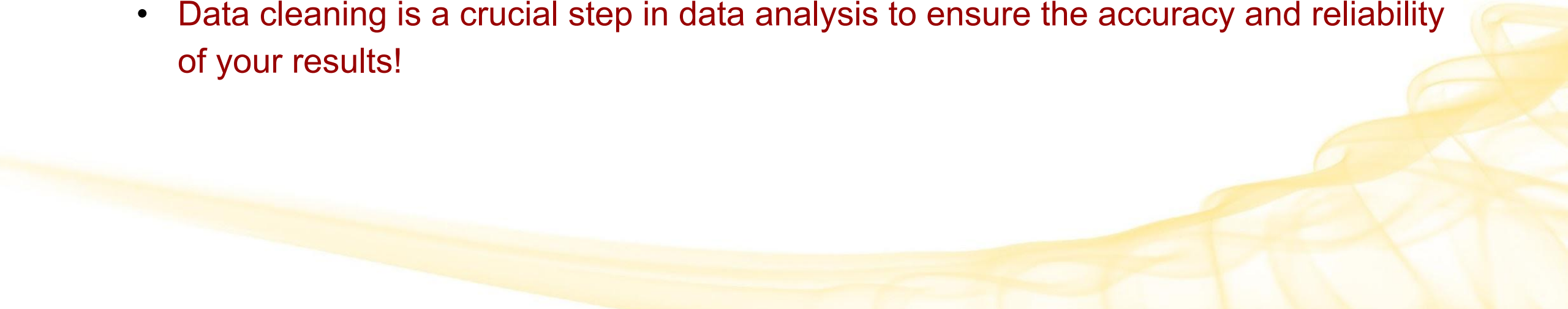
Data Pre-Processing Summary

- **Dirty Data:** Data can be dirty due to missing values, column combinations, or joining multiple tables.
 - **Data Cleaning:** Before doing data analysis you must clean the data to ensure it's accurate and ready for analysis.
 - **Handling Obvious Issues:** Use functions like `apply()` for simple and obvious data issues, like correcting misspellings in categorical data.
 - **Data Typing:** Ensure data is in the correct format; use `astype()` or `apply()` for type conversions.
 - **Merging Data:** Merge data/tables carefully, ensuring formats match.
 - **Evolving Labeling Schemes:** Adapt to changes in labeling schemes by dividing the dataset, inferring old values, or handling changes as needed.
- 

Data Pre-Processing Summary

- **Dealing with Duplicates:** Identify and potentially remove exact or subtle duplicates to maintain data quality.
 - **Outlier Detection:** Use methods like z-scores to find and consider removing outliers.
 - **Imputing Missing Data:** Handle missing data using simple methods like single imputation, or more advanced methods like Bayesian imputation.
 - **Boundary Conditions:** Be aware of data limits imposed by instruments, databases, or surveys and adjust analysis accordingly.
 - **Irrelevant Data:** Identify and drop any data that clearly does not contribute to your analysis goal.
- 

Data Pre-Processing Summary

- **Detecting Incorrect Data:** Identify issues like attractors, discontinuities, strange modes, and data outside valid bounds.
 - **Repairing Instrument Errors:** Correct data from malfunctioning sensors based on past data or similar sensors.
 - **Signs of Boundary Conditions:** Look for sudden changes or gaps in data and observe patterns in visualizations.
 - **Data cleaning is a crucial step in data analysis to ensure the accuracy and reliability of your results!**
- 

Demo

